Assignment 3

INF-2300

Erling Heimstad Willassen

1. Introduction

To transfer data from an application on a computer to another computer over the internet, the data must go through the transport and network layer. In the transport layer, the data is processed as a package to be further transitioned into the network layer. Since the internet is constricted, packages can be dropped, delayed and even corrupted in the transit from one place to another. In this assignment, the student is tasked with processing these constrictions so that all packages are sent and delivered correctly through a network simulation.

2. Technical Background

The ISO model is a figurative model that displays the many layers of data network communication from the highest level of application layer to the lowest of the physical layer [1]. In all of these layers there are different protocols that dictate how the data should be processed to be sent and received correctly in the other end. By having universal protocols of how data should be processed in each of these layers, data can be sent through a network all across the globe without the fear of not being processed correctly in the receiving end.

In the highest level of application layer, the data from an application is being correctly processed by protocols before being sent to the transport layer. In the transport layer, the data will be placed in several packets before being sent to the network layer. On the receiving end of the data transfer, the packets can arrive out of order due to network routing, corrupted from radiation, or dropped completely due to router buffers not having enough memory for incoming packets. The quality, order and acknowledgment of these packets must be clarified before being sent up to the application layer.

The transport layer have the task of flow control, error control, and acknowledgment of packets from a host to a client in a network [1]. Data is divided and sent as several packets of the complete data from the host to the client. On the client side, these packets must be processed correctly in the transport layer so that the data from the host application layer is received in the same state to the client application layer. This can be done by adding attributes to the packets from the host side such as a checksum and a sequence number.

The checksum is a code that can be used to check for data integrity in a packet. The checksum can be calculated by using a hash-function on the data being sent in the packet [2]. On the receiving end, the same hash calculation is done on the data and compared to the packet's checksum. If the checksum is the same, the data was most likely unaltered throughout the network, and if not, the data is most likely corrupted and is not in the same state as from the host. This way, the checksum

acts as a fingerprint of the data and can be used to check if the data became corrupted on the way from the host to the client.

A sequence number on the packet tells the client in which order the packet should be in the receiving end. A packet can be delayed or even dropped throughout the journey in a network, and therefore not come in the correct order on the client side as it was sent from the host. By having a sequence number, a packet arriving on the client can therefore be correctly processed before being sent up the application layer without sending packet data in an incorrect order and creating problems.

To confirm that packets are received on the client side, an acknowledgment from the client transport layer is sent back to the host transport layer. By doing this, the host will understand that a specific packet is correctly received on the client side, and does not need to resend the packet to the client. If the host does not receive any acknowledgments or receiving a non-acknowledgment (a corrupted packet), the host will resend the packet after a specific time interval. This way, if a packet is being dropped or corrupted through the network, the host will resend the packet until it has been confirmed that the client has received the packet in a correct state.

Go-Back-N (GBN) is an algorithm which provides reliable data transfer of sending and receiving packets from host to a client [3]. In GBN, a window containing a certain number packets are created on the host side to be sent to the client. A timer would also be initiated that is connected to the window. When the timer is up, the timer will initiate a function that resend the packet window. This is because if the initial sending of packets are dropped or delayed, the same packet window will be resent to the client instead of waiting for acknowledgment that may or not happen. If an acknowledgment is received to the host, that specific packet is removed from the packet window and a new packet is added. With this algorithm, a much more effective way of sending packets is achieved than to that of sending one and one packet and waiting for acknowledgment before continuing. GBN allows to send several packets at the same time, but not all packets that could over-flood the client side.

3. Design

A packet that will be sent from the host to the client will consist of three attributes: the data, checksum and sequential number. The data will be segmented into several data partitions where each partition will be placed into its own packet. The checksum for each data partition will be calculated and placed into its respective packet. The order of the data will be also placed into the correct packet based on its partitioned data as a sequence number. After a packet is being created, the packet is placed in a packet window and sent. If the packet window is full, the packet will be added to a packet buffer list instead.

The transport layer will have two main functions: one for sending and for receiving packets. The sender function will divide data into segments and assign each to a packet, where checksum and sequence number will be sequentially added to its respective packet. The first packets that are added to the packet window will be sent, whereas the others will be added to a buffer list. In the receiving function will be divided into two parts, one for data packets (client) and one for acknowledgment packets (host). If data packet, the packet checksum and the calculated checksum will be calculated to check for corruption. If corrupted, a non-acknowledgment packet will be sent back to the host for

resending that specific packet again. If not, the sequence number of that packet will be checked and compared to the last sent packet to application layer of the client. If the sequence number is the next one in line to the application layer, the packet will be sent. And if not, the client will ignore the packet. For the acknowledgment part of the function, the packet will be checked if its an acknowledged packet (ACK) or a non-acknowledgment packet (NACK). If ACK, the host will remove that specific packet that is acknowledged from the packet window and a new packet from the packet buffer will be added. If a NACK, the host will resend the packet that was non-acknowledged by the client.

4. Implementation

The code is implemented in python with a pre-code for the simulation. The task handed to the student was to implement the code for the packet and for the transport layer.

The packet was created as a class object where attributes or data, sequence number and checksum was added. Since the simulation required data to be sent through the network layer, an ACK attribute was given to the packet class as well. If the ACK attribute was None, it meant the packet was a data packet. If the ACK attribute was set as True, the packet acts as an ACK when received by the host. If ACK attribute was set as False, the packet acts as a NACK. The sequence number for that specific ACK or NACK would be the same as for the acknowledged packet on the client side, so the host will understand which packet the ACK or NACK was sent for.

For the transport layer, one had to implement both sending and receiving packets. For sending packets, the simulation gave the segments for the total data sequentially. This would mean that it was not necessary to divided the total data into segments as explained in the design chapter. The segmented data was also already encoded, so need to further modify the data before applied to a packet object. A counter for the host was initiated where for each data segment that was received was counted, and given as a sequence number to the number attribute to the packet. This way, the sequentially order of the data segments from the simulator was in tight control. Then, the checksum for the data was calculated and written into the checksum attribute. The ACK attribute was set as None from initiation, indicating that the packet would be a data packet. After creating the packet, the packet was added to the window list as long as the length of list was less then the given size of the window (in the code, maximum of five packets). Else, the packet would be added to another list as packet buffer. After adding the packet to the window list, the packet would be sent to the client. Right after the sending of packet, a timer object would be initiated set to a specific time. If the timer time was up, another function would be initiated where it would iterate through all packets in the window list and be sent all over again. After sending all the packets in the window, the timer would be reinitialized for another round.

On the receiving end of the packet, the packet attribute of ACK would be checked. Here, three instances could happen after checking for ACK attribute:

ACK attribute	Implementation information
None	If the ACK packet attribute was checked to be None, this would mean for the client side that the packet is a data packet. When that information is achieved, the checksum would be calculated for that data packet. Here, the function would be divided into two parts. • If the checksum would not be equal, the packet would not be further processed due to the chance of data corruption. If corrupted, a copy of the packet would be created and the attribute ACK would be changed to False to indicate a NACK packet. This packet would be sent back to the host, telling the host that this specific packet needs to sent again to the client. • If the checksum comparison would equal to be the same, the data packet would be further processed by comparing the sequence number of the received packet to the sequence number that was last sent up to the application layer. If the received packet was next in line to be sent to the application layer, the data would be sent upwards and a copy of that packet would be created with the ACK attribute set to True, and sent back to the host. If the packet was not the next in line, the client would not send anything back.
True	If the ACK attribute was checked to beTrue, it would mean to the host and client that the packet would be an ACK packet. This packet would be further processed by the host, where the sequence number of the received ACK packet would be checked against the sequence number of all the packets in the window list. If one packet was a match, that packet would be removed from the window list. If not a match, this would mean a previously received ACK had already deleted it from the list and nothing would be done. Further, since there is room in the window list due to deleting a packet from that list, a new data packet is added to the window list from the packet buffer list (if packets in buffer list). This would always keep the window list in maximum capacity until there are no more packets in the buffer list.
False	If the ACK attribute was checked to be False, it would mean that the received packet was a NACK packet. This would indicate the client received a packet that was corrupted. The sequence number of the NACK packet would then be compared to all the packets in the window list. If a match, then that packet would be sent again to the client. If not, then nothing would be sent as that would indicate a previous ACK was received for that specific packet.

This implementation worked nicely and managed to send all packets in correct order to the application layer. However, sometimes sending an ACK from the client back to the host was dropped when traversing through the network. This means that the data packet was processed correctly, but the ACK was never received by the host. Since ACK's would only be sent when a data packet is processed in a sequential order, the client would never send an ACK back again to confirm the data packet. This would make the host continuing sending the data packet and the client continuously ignoring the packet. Even after the simulation was finished, the ignored data packets was still being sent from the host. This resulted in an if statement after confirming that the ACK

attribute was None and checksum was correct, where if the sequence number was less than the most recent sequence number data packet sent to application layer would initiate sending an ACK back to the host. This way, the host would only receive ACK's for previously processed data packets by the client and not for those who haven't been processed yet. This would result in that previously processed packets would in the end receive an ACK and new data packets could be added to the window list from the buffer list, solving the eternal loop of resending data packets that had been processed.

To test the performance of the implementation, the changing of chance values for dropping, corrupting and delaying for a packet would be performed. In the experiments, the packet size and number of packets for simulation were 4 and 10, respectively, and held low so the simulation would not take too long time.

5. Results

The simulation had the possibility to alter the chances of dropping, corrupting and delaying a packet. These values would be used to test the implementation in how well it handles the different cases. The number of packets to be sent was set to 10 and the packet size to 4, and would be constant throughout the experiments. For the delay, the delay amount was set constant to 0.5.

*Table 1: Code implementation experimented against various packet modification and the chances of its occurrence. Other constants were packet size of 4 and packet amount of 10. *Delay_time for delay_chance was set as 0.5.*

Chance value	**0.0** (0 %)	**0.1** (10 %)	**0.9** (90 %)
Drop_chance	0.012233 s	1.25774 s	57.63475 s
Corrupt_chance	0.012233 s	0.01678 s	0.06840 s
*Delay_chance**	0.012233 s	0.91494 s	2.07559 s
All chances	0.012233 s	3.62637 s	332.96210 s

6. Discussion

From the results in table 1, one can see that the implementation worked against all packet occurrences such as dropping, corruption and delay with various results. From all the different problems a packet can be affected by the dropping had the most significant time use on the implementation in both chances of 10 % and 90 %. At 10 % chance, the drop problem had an time increase of almost 100x and 4700x at 90 % chance. Delay had the next highest time use on the implementation in both chances, with 75x increase at 10 % chance and 170x increase at 90 %. The problem with the least amount of time increase was corruption, where it only had a time increase of 1.4x at 10 % chance and a 6x increase at 90 % chance.

The reason for why packet drop problem had the highest time increase of all the problems are most likely not just one, but many. On the statistical side, a dropped packet can happen two ways; both from host to client, and client to host. This means this problem affects twice in clients receiving

packets and host receiving ACKs or NACKs about the received packets. At 90 % chance most of the packets are being dropped, and therefore few packets are being received on both ends. Of the packets that are received, the ACKs or NACKs that are sent back are most likely also dropped and therefor occupying space in the window list on the host side. Even if many packets have been processed on the client side, many of the same packets are most likely still sent from the host window list since few to no ACKs are received to the host. Another major factor is also the implementation. The window list is static and was set to contain five packet at a time. This could be maybe to small for the chance value. The timer could also been set to a lower value to send the missing packets more often, and therefore increasing the chances that some packets are received on the other side. The biggest reason could be that of how the client handles packets that are not in order. If the client receives a packet that is not the next packet to be sent to the application layer, the received packet is ignored and dropped. If the client side had a buffer list to save all packets that are not in order, the client could be using those packets on a later stage for when those packets can be sent to the application layer. This way, data packets does not need be sent from the host and received by the client at the right time to be further processed to the application layer. This would make a host client communication a much faster and effective affair with much less sending of packets from the host. If implementing a system like this, the time usage would most likely go way down.

On the other hand, a 90 % chance for any problems to packets are quite high and not a very realistic occurrence in a normal everyday network. However, still at 10 % chance of any problems except for corruption still has a time increase of almost 100x. These time usages are most likely not realistic either as the simulation could use much time on printing the happenings of the simulation and other object handling that would not occur in a normal network communication. Still, these findings can give a small insight on which problems have the biggest affect on time usage in network communication of packets.

7. Conclusion

Computer communication through a network can give rise to many problems such as packet drop, delay and corruption. At certain chances of these problems, they could give a severe increase of time use. By handling these at the transport layer with Go-Back-N algorithm on the host side and sending ACK's/NACK's from the client, these problems can be succumbed. The student accomplished the task at hand from the assignment where packets could be sent and received correctly between a host and a client.

8. Sources

[1] Wikipedia Contributors, “OSI model,” *Wikipedia*, Mar. 11, 2019. https://en.wikipedia.org/wiki/OSI_model

[2] Wikipedia Contributers, “Checksum,” *Wikipedia*, Jan. 07, 2020. https://en.wikipedia.org/wiki/Checksum

[3] Wikipedia Contributers, “Go-Back-N ARQ,” *Wikipedia*, Feb. 16, 2020. https://en.wikipedia.org/wiki/Go-Back-N_ARQ